SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

SIMATS ENGINEERING - CSA17 ARTIFICIAL INTELLIGENCE LAB

Day 1 - Program 5	Course Outcome: CO3	Language: Python 3

Question

Write the python program for Missionaries Cannibal problem

Aim

To move 3 missionaries and 3 cannibals across a river using a boat that carries at most 2 people, so that cannibals never outnumber missionaries on either bank.

Algorithm / Explanation

A state is (M_left, C_left, boat_side). BFS generates the 5 possible boat loads (1M, 2M, 1C, 2C, 1M+1C) from each state.

States that violate the safety rule on either bank are discarded. BFS gives the minimum number of crossings (11).

Program

```
# Missionaries and Cannibals Problem using BFS
from collections import deque

# State = (missionaries on left, cannibals on left, boat side: 0 = left, 1 = right)
START, GOAL = (3, 3, 0), (0, 0, 1)
MOVES = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]     # (missionaries, cannibals) in boat

def valid(m, c):
    if not (0 <= m <= 3 and 0 <= c <= 3):
        return False
    if m > 0 and c > m:                  # left bank: cannibals outnumber missionaries
        return False
    if (3 - m) > 0 and (3 - c) > (3 - m):  # right bank
        return False
    return True

def solve():
    queue = deque([(START, [START])])
    seen = {START}
    while queue:
        (m, c, b), path = queue.popleft()
        if (m, c, b) == GOAL:
            return path
        for dm, dc in MOVES:
            nm, nc = (m - dm, c - dc) if b == 0 else (m + dm, c + dc)
            ns = (nm, nc, 1 - b)
            if valid(nm, nc) and ns not in seen:
                seen.add(ns)
                queue.append((ns, path + [ns]))

path = solve()
print("Solution in", len(path) - 1, "crossings")
print("(M_left, C_left, boat)  ->  boat: 0 = left bank, 1 = right bank\n")
for i, s in enumerate(path):
    print(f"Step {i}: {s}")
```

Output

```
Solution in 11 crossings
(M_left, C_left, boat)  ->  boat: 0 = left bank, 1 = right bank

Step 0: (3, 3, 0)
Step 1: (3, 1, 1)
Step 2: (3, 2, 0)
```

```
Step 3: (3, 0, 1)
Step 4: (3, 1, 0)
Step 5: (1, 1, 1)
Step 6: (2, 2, 0)
Step 7: (0, 2, 1)
Step 8: (0, 3, 0)
Step 9: (0, 1, 1)
Step 10: (1, 1, 0)
Step 11: (0, 0, 1)
```

Result

Thus the Python program was executed successfully and the output was verified.